



طراحی و پیاده سازی پردازنده تک چرخه ۳۲ بیتی

معماری کامپیوتر – گزارش کار تمرین عملی پنجم

استاد

دکتر سربازی آزاد

دانشگاه صنعتی شریف

بهار ۱۴۰۵

محمد مهدی مرادی – ۴۰۳۱۰۶۶۸۱

۳	مقدمه
۳	۱- طراحی مسیر داده (Datapath) و مدیریت آدرس‌ها
۳	۲- واحد حساب و منطق (ALU) و محاسبات بدون عملگر
۷	۳- بانک ثبات‌ها (Register File)
۸	۴- واحد کنترل (Control Unit) و تولید سیگنال‌ها
۱۱	۵- منطق پرش‌ها (Branch & Jumps)
۱۱	۶- رفع چالش‌های محیط شبیه‌ساز (Testbench)
۱۲	نتیجه گیری

هدف از این پروژه، طراحی و پیاده‌سازی یک پردازنده ۳۲ بیتی تک‌چرخه (Single-Cycle) مبتنی بر معماری MIPS با استفاده از زبان توصیف سخت‌افزار Verilog بود. این پردازنده باید قادر به اجرای مجموعه‌ای از دستورات محاسباتی (R-Type)، انتقال داده و منطقی (I-Type) و پرش‌ها (J-Type) می‌بود. تمامی قطعات از پایه طراحی شدند و در نهایت پردازنده توانست برنامه‌های تست (مانند محاسبه دنباله فیبوناچی و تبدیل مبنای ۳) را با موفقیت و بدون خطا اجرا کند.

در ادامه، جزئیات پیاده‌سازی هر بخش و نحوه استخراج منطق آن‌ها بر اساس خواسته‌های صورت پروژه آورده شده است.

۱- طراحی مسیر داده (Datapath) و مدیریت آدرس‌ها

مسیر داده به گونه‌ای طراحی شد که تمام مراحل خواندن دستور، رمزگشایی، اجرا، دسترسی به حافظه و بازنویسی در یک چرخه ساعت (Clock Cycle) انجام شود. یکی از نکات کلیدی صورت پروژه، نحوه ارتباط با حافظه دستورات (Instruction Memory) و حافظه داده (Data Memory) بود. از آنجایی که شمارنده برنامه (PC) در معماری‌های ۳۲ بیتی به صورت بایت‌محور (Byte-addressable) کار می‌کند و در هر مرحله ۴ واحد افزایش می‌یابد ($PC+4$)، اما حافظه‌های تست‌بنچ کلمه‌محور (Word-addressable) بودند، ما در ماژول اصلی ۲ بیت کم‌ارزش آدرس‌ها را نادیده گرفتیم تا آدرس‌دهی دقیقاً مضرب ۴ باقی بماند (استفاده از $PC[31:2]$).

۲- واحد حساب و منطق (ALU) و محاسبات بدون عملگر

طبق محدودیت پروژه، استفاده از عملگرهای ترکیبی ضرب و تقسیم مجاز نبود. برای حل این مسئله، ماژول‌های ریاضی به صورت ماشین‌های حالت (State Machine) کلاک‌دار طراحی شدند که عملیات را در ۳۲ چرخه کلاک انجام می‌دهند:

- **ماژول ضرب (MUL):** با استفاده از الگوریتم ترتیبی «شیفت و جمع» پیاده‌سازی شد. با دریافت سیگنال Start، ماژول در هر کلاک یک بیت از مقسوم‌علیه را بررسی کرده، در صورت لزوم عملوند را جمع و سپس شیفت می‌دهد. پس از ۳۲ کلاک، سیگنال Ready صادر می‌شود.

- **ماژول تقسیم (DIV):** بر اساس الگوریتم «تقسیم بازیاب» (Restoring Division) طراحی شد. ماژول در هر چرخه کلاک، عملیات شیفت به چپ مقسوم و تفریق آزمایشی از مقسوم‌علیه را انجام می‌دهد و بیت‌های خارج‌قسمت را یک‌به‌یک تولید می‌کند.

- **کنترل‌کننده ALU:** این واحد وظیفه اجرای عملیات ساده (جمع و تفریق) در یک کلاک و همچنین مدیریت ماژول‌های ضرب و تقسیم را بر عهده دارد. برای جلوگیری از ریست شدن مداوم ماشین‌های حالت، این کنترل‌کننده با استفاده از تکنیک لبه‌گیری (ذخیره وضعیت کلاک قبلی دستور)، با تشخیص دستورات MUL یا DIV، یک پالس Start تنها به مدت یک چرخه کلاک برای ماژول مربوطه ارسال کرده و سپس پردازنده منتظر دریافت سیگنال Ready می‌ماند.

از آنجا که محاسبات ضرب و تقسیم ۳۲ کلاک به طول می‌انجامند، پردازنده باید تا زمان آماده شدن نتیجه متوقف شود. برای این منظور سیگنال `alu_ready` در مسیر داده اصلی (Main) اعمال شد:

• توقف PC: ثبات شمارنده برنامه تنها در صورتی با آدرس دستور بعدی به‌روز می‌شود که سیگنال `alu_ready` فعال باشد.

• محافظت از حافظه و ثبات‌ها: برای جلوگیری از نوشتن داده‌های نامعتبر در طول ۳۲ چرخه انتظار، سیگنال‌های `MemWrite` و `RegWrite` با سیگنال `alu_ready` به صورت منطقی AND شدند.

```
module ALU(  
    input wire clk,  
    input wire [1:0] ALUOp,  
    input wire [31:0] Reg1, Reg2,  
    output reg [31:0] Result,  
    output wire zero,  
    output wire ready  
);  
  
wire [31:0] mul_res, div_res;  
wire mul_ready, div_ready;  
reg prev_mul, prev_div;  
  
initial begin  
    prev_mul = 1'b0;  
    prev_div = 1'b0;  
end  
  
always @(posedge clk) begin  
    prev_mul <= (ALUOp == 2'b10);  
    prev_div <= (ALUOp == 2'b11);  
end  
  
wire start_mul = (ALUOp == 2'b10) && !prev_mul;  
wire start_div = (ALUOp == 2'b11) && !prev_div;  
  
MUL mul(clk, start_mul, Reg1, Reg2, mul_res, mul_ready);  
DIV div(clk, start_div, Reg1, Reg2, div_res, div_ready);  
  
always @(*) begin  
    case (ALUOp)  
        2'b00: Result = Reg1 + Reg2;  
        2'b01: Result = Reg1 - Reg2;  
        2'b10: Result = mul_res;  
        2'b11: Result = div_res;  
        default: Result = 32'b0;  
    endcase  
end  
  
assign zero = (Result == 32'b0) ? 1'b1 : 1'b0;  
assign ready = (ALUOp == 2'b10) ? mul_ready :  
                (ALUOp == 2'b11) ? div_ready : 1'b1;  
endmodule
```

```

module MUL(
    input wire clk,
    input wire start,
    input wire [31:0] Reg1, Reg2,
    output reg [31:0] Result,
    output wire ready
);
    reg [5:0] count;
    reg [31:0] a, b;
    reg busy;

    initial busy = 1'b0;

    assign ready = ~busy & ~start;

    always @(posedge clk) begin
        if (start) begin
            a <= Reg1;
            b <= Reg2;
            Result <= 32'b0;
            count <= 6'b0;
            busy <= 1'b1;
        end else if (busy) begin
            if (a[0]) Result <= Result + b;
            a <= a >> 1;
            b <= b << 1;
            count <= count + 1;
            if (count == 31) busy <= 1'b0;
        end
    end
endmodule

```

```

module DIV(
    input wire clk,
    input wire start,
    input wire [31:0] Reg1, Reg2,
    output reg [31:0] Result,
    output wire ready
);
    reg [5:0] count;
    reg [31:0] q, r, b;
    reg busy;

    initial busy = 1'b0;

    assign ready = ~busy & ~start;

    always @(posedge clk) begin
        if (start) begin
            q <= Reg1;
            b <= Reg2;
            r <= 32'b0;
            Result <= 32'b0;
            count <= 32;
            busy <= 1'b1;
        end else if (busy) begin
            if ({r[30:0], q[31]} >= b) begin
                r <= {r[30:0], q[31]} - b;
                Result[count - 1] <= 1'b1;
            end else begin
                r <= {r[30:0], q[31]};
                Result[count - 1] <= 1'b0;
            end
            q <= q << 1;
            count <= count - 1;
            if (count == 1) busy <= 1'b0;
        end
    end
endmodule

```

۳- بانک ثبات‌ها (Register File)

بانک ثبات‌ها شامل ۳۲ ثبات ۳۲ بیتی با دو پورت خواندن (ترکیبی) و یک پورت نوشتن (ترتیبی) طراحی شد. برای رعایت دقیق قواعد معماری خواسته شده، دو مکانیزم مهم در این ماژول اعمال شد:

- محافظت از ثبات صفر: ثبات $0\$$ همیشه باید مقدار صفر را برگرداند. برای تضمین این موضوع، در بلوک نوشتن شرط گذاشتیم که آدرس مقصد 0 نباشد.

- بازنشانی (Reset): با دریافت سیگنال بازنشانی به صورت آسنکرون، تمام ۳۲ ثبات در یک بلوک **always** صفر می‌شوند تا پردازنده در حالت پایدار شروع به کار کند.


```

module RF(
    input wire Clk, Rst, RegWrite,
    input wire [4:0] ReadReg1, ReadReg2, WriteReg,
    input wire [31:0] Data,
    output wire [31:0] ReadData1, ReadData2,
    output wire [31:0] R0, R1, R2, R3, R4, R5, R6,
    R7, R8, R9, R10, R11, R12, R13, R14, R15,
    R16, R17, R18, R19, R20, R21, R22, R23,
    R24, R25, R26, R27, R28, R29, R30, R31
);
    reg [31:0] Registers [0:31];
    integer i;

    initial begin
        for (i = 0; i < 32; i = i + 1) begin
            Registers[i] = 32'b0;
        end
    end

    assign ReadData1 = (ReadReg1 == 5'd0) ? 32'b0 : Registers[ReadReg1];
    assign ReadData2 = (ReadReg2 == 5'd0) ? 32'b0 : Registers[ReadReg2];

    assign {R31, R30, R29, R28, R27, R26, R25, R24, R23, R22, R21, R20, R19, R18, R17, R16,
        R15, R14, R13, R12, R11, R10, R9, R8, R7, R6, R5, R4, R3, R2, R1, R0} =
        {Registers[31], Registers[30], Registers[29], Registers[28], Registers[27],
        Registers[26], Registers[25], Registers[24], Registers[23], Registers[22],
        Registers[21], Registers[20], Registers[19], Registers[18], Registers[17],
        Registers[16], Registers[15], Registers[14], Registers[13], Registers[12],
        Registers[11], Registers[10], Registers[9], Registers[8], Registers[7],
        Registers[6], Registers[5], Registers[4], Registers[3], Registers[2],
        Registers[1], Registers[0]};

    always @(posedge Clk or posedge Rst) begin
        if (Rst == 1'b1) begin
            for (i = 0; i < 32; i = i + 1) begin
                Registers[i] <= 32'b0;
            end
        end else if (RegWrite == 1'b1 && WriteReg != 5'b00000) begin
            Registers[WriteReg] <= Data;
        end
    end
endmodule

```

۴- واحد کنترل (Control Unit) و تولید سیگنال‌ها

واحد کنترل به عنوان مغز پردازنده، ورودی‌های opcode (۶ بیت با ارزش) و funct (۶ بیت کم‌ارزش) را دریافت کرده و مسیر داده‌ها را کنترل می‌کند. سیگنال‌ها به این شکل استخراج و اعمال شدند:

- سیگنال RegDst: برای تعیین مقصد نوشتن طراحی شد. مقدار ۰ دستورات R-Type (مقصد ثابت rd) مقدار ۱ برای I-Type (مقصد ثابت rt)، و مقدار ۲ برای دستورات JAL (مقصد ثابت ۳۱) در نظر گرفته شد.

• سیگنال ALUSrc: انتخاب می‌کند که عملوند دوم ALU از ثبات خوانده شود (برای R-Type ها) یا مقدار ۱۶ بیتی اکستندشده دستور باشد (برای I-Type ها).

• سیگنال MemtoReg: یک مالتی‌پلکسر ۴ ورودی طراحی شد. خروجی ۰ برای نتایج ALU، خروجی ۱ برای خواندن از حافظه (LW)، خروجی ۲ برای نوشتن PC+۴ در دستور JAL، و خروجی ۳ برای یک ترفند خاص در دستور LUI اختصاص یافت.

• مدیریت دستور LUI: از آنجا که ALU جمع و تفریق انجام می‌دهد و انتقال مستقیم مقدار ۱۶ بیتی (به همراه ۱۶ صفر در سمت راست) از طریق آن دشوار بود، ما مقدار شیفت‌خورده را مستقیماً به عنوان یک ورودی جدید به مالتی‌پلکسر MemtoReg دادیم تا بدون عبور از ALU مستقیماً در ثبات نوشته شود.

```

module CU(
    input wire [5:0] opcode, funct,
    output reg [1:0] RegDst, MemtoReg, ALUOp,
    output reg RegWrite, ALUSrc, MemRead, MemWrite, Branch, Jump, Jal, Jr
);
    always @(*) begin
        RegDst = 2'b00;
        MemtoReg = 2'b00;
        ALUOp = 2'b00;
        RegWrite = 1'b0;
        ALUSrc = 1'b0;
        MemRead = 1'b0;
        MemWrite = 1'b0;
        Branch = 1'b0;
        Jump = 1'b0;
        Jal = 1'b0;
        Jr = 1'b0;

        case(opcode)
            6'b000000:
                //JR
                if(funct == 6'b001000) Jr = 1'b1;
                else begin
                    RegWrite = 1'b1;
                    case(funct)
                        //ADD
                        6'b100000: ALUOp = 2'b00;
                        //SUB
                        6'b100010: ALUOp = 2'b01;
                        //MUL
                        6'b011000: ALUOp = 2'b10;
                        //DIV
                        6'b011010: ALUOp = 2'b11;
                        default: ;
                    endcase
                end

            //LW
            6'b100011: begin ALUSrc = 1'b1; MemtoReg = 2'b01; MemRead = 1'b1; ALUOp = 2'b00; RegWrite = 1'b1; RegDst = 2'b01; end
            // SW
            6'b101011: begin ALUSrc = 1'b1; MemWrite = 1'b1; ALUOp = 2'b00; RegDst = 2'b01; end
            // ADDI
            6'b001000: begin ALUSrc = 1'b1; ALUOp = 2'b00; RegWrite = 1'b1; RegDst = 2'b01; end
            // SUBI
            6'b001001: begin ALUSrc = 1'b1; ALUOp = 2'b01; RegWrite = 1'b1; RegDst = 2'b01; end
            // LUI
            6'b001111: begin ALUSrc = 1'b1; RegWrite = 1'b1; RegDst = 2'b01; MemtoReg = 2'b11; end
            // J
            6'b000010: begin Jump = 1'b1; end
            // JAL
            6'b000011: begin MemtoReg = 2'b10; Jump = 1'b1; Jal = 1'b1; RegWrite = 1'b1; RegDst = 2'b10; end
            // BEQ
            6'b000100: begin Branch = 1'b1; ALUOp = 2'b01; end

            default: ;
        endcase
    end
endmodule

```

۵- منطق پرش‌ها (Branch & Jumps)

یکی از بخش‌های پیچیده پروژه، مدیریت سه نوع پرش مختلف بود که در ماژول main پیاده‌سازی شدند:

- پرش شرطی (BEQ): ابتدا مقدار ۱۶ بیتی immediate علامت‌دار (Sign-extend) شد، سپس ۲ بیت به چپ شیفت داده شد و با $PC+4$ جمع گردید. اگر خروجی zero از ALU فعال بود و سیگنال Branch نیز ۱ بود، این آدرس در PC قرار می‌گرفت.

- پرش مطلق (JAL): مقدار ۲۶ بیتی آدرس پرش در دستور، به عنوان ورودی دریافت شد و پس از ۲ بیت شیفت به چپ، با ۴ بیت باارزش $PC+4$ ترکیب شد تا آدرس نهایی ساخته شود. در دستور JAL، علاوه بر پرش، مقدار $PC+4$ مستقیماً در ثبات شماره ۳۱ ذخیره شد.

- پرش به ثبات (JR): با اضافه کردن یک سیگنال کنترلی جدید (Jr)، مسیر داده‌ای تعبیه شد که مقدار خوانده شده از ثبات rs را مستقیماً و بدون هیچ تغییری به عنوان آدرس دستور بعدی وارد PC کند.

```
wire [31:0] branch_target = pc_plus_4 + (sign_ext_imm << 2);
wire [31:0] jump_target = {pc_plus_4[31:28], addr, 2'b00};
wire do_branch = Branch & alu_zero;

wire [31:0] pc_after_branch = do_branch ? branch_target : pc_plus_4;
wire [31:0] pc_after_jump = Jump ? jump_target : pc_after_branch;

assign next_pc = Jr ? read_data1 : pc_after_jump;
```

۶- رفع چالش‌های محیط شبیه‌ساز (Testbench)

در حین اتصال پردازنده به تست‌بنچ از پیش نوشته شده، دو چالش اصلی در محیط شبیه‌سازی رخ داد که با راهکارهای مهندسی در سخت‌افزار برطرف شدند:

- جلوگیری از خطاهای Invalid Address: تست‌بنچ به تغییرات پورت DataAddr حساس بود و از آنجا که در دستورات محاسباتی، خروجی ALU به این پورت متصل بود، اخطارهای متعددی تولید می‌شد. برای رفع این مشکل، در ماژول اصلی شرطی قرار دادیم که آدرس واقعی تنها در صورت فعال بودن خواندن یا نوشتن در حافظه (MemRead | MemWrite) ارسال شود و در غیر این صورت یک آدرس بی‌اثر و امن (مثلاً ۰X۴۰۰) ارسال شود.

- رفع گره اجرای دستور اول (Race Condition): برای بیدار کردن تست‌بنچ در کلاک اول (Step ۰) و جلوگیری از خوانده شدن دستور توخالی، شمارنده برنامه (PC) به جای عدد صفر، با مقدار آدرس انتهای حافظه (۲۰۴۴) مقداردهی اولیه شد تا به محض دریافت سیگنال بازنشانی (Rst)، تغییر آدرس از ۲۰۴۴ به ۰ باعث تحریک تست‌بنچ و بارگذاری صحیح دستور اول شود.

نتیجه گیری

طراحی به صورت ماژولار (تفکیک به واحدهای ALU, MUL, DIV, CU, RF و Main) باعث شد تا کدها کاملاً خوانا، قابل عیب‌یابی و منطبق بر اصول سینتکس وریلاگ تولید شوند. در نهایت سیستم توانست محاسبات فیبوناچی، اجرای صحیح عملیات ضرب بدون استفاده از عملگر مستقیم و الگوریتم تبدیل مبنای ۳ را با دریافت تاییدیه نهایی (ACCEPTED) به اتمام برساند.

```
WARNING: Invalid address to read : xxxxxxxX
WARNING: Invalid address to read : 00000000
WARNING: Invalid address to read : 00000034
WARNING: Invalid address to read : 00000000
WARNING: Invalid address to read : 0000263c
WARNING: Invalid address to read : 00000000
WARNING: Invalid address to read : 00000cbc
WARNING: Invalid address to read : 00000000
WARNING: Invalid address to read : 0000043c
WARNING: Invalid address to read : 00000000
WARNING: Invalid address to read : 00000168
WARNING: Invalid address to read : 00000000
WARNING: Invalid address to read : 00000078
WARNING: Invalid address to read : 00000000
WARNING: Invalid address to read : 00000024
WARNING: Invalid address to read : 0000000c
WARNING: Invalid address to read : 00000000
fib(10) = exp: 55  real: 55
fib(11) = exp: 89  real: 89
product = exp: 4895  real: 4895
base3 digits LSD first:
2 2 0 1 0 2 0 2
base3 = 20201022
ACCEPTED
208 / 208
HW5/tb.v:324: $finish called at 424 (1s)
```